

# Initialisation and reproducibility

Project: Image-pattern Network Investigation

## 1. Your mission

### Your learning target

I can trace how starting values affect a fixed training budget.

I can explain a simple hidden-unit symmetry.

I can record enough information to replay and audit a classroom experiment.

### Remember before reading

Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.

Without opening the old lesson, explain one key idea from Regularisation and early stopping. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Learning rate experiments. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Updating parameters. Give an example and a mistake that the idea helps you avoid.

After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the evidence shows was wrong.

This week you will practise:

Trace how starting values affect a fixed training budget

Explain a simple hidden-unit symmetry

Record enough information to replay and audit a classroom experiment

### Retrieve what you already know

Why must an early-stopping report identify both the stop epoch and the restored checkpoint?

#### My starting explanation

---

---

---

## This week's end goal

By the end, I can explain initialisation and reproducibility using a traced example and a stated limitation.

## 2. Notice the evidence

Return to  $h = \text{ReLU}(x)$ ,  $y = vh$ , training pairs (1, 2) and (2, 4), ordinary MSE and a frozen hidden unit. Only  $v$  is trained. Keep full-data updates, learning rate 0.1 and exactly two updates. Compare initial  $v$  values 0, 1 and 3. This experiment changes only the start.

The gradient is  $5(v - 2)$ , so each step gives new  $v = 0.5v + 1$ . Signed distance from 2 halves on every update. Different starting distances therefore produce different losses after this finite budget, even though all three runs approach the same minimum if this exact procedure continues.

**Initial v · After one update · After two updates · Final training MSE**

**Initial v:** 0

**After one update:** 1

**After two updates:** 1.5

**Final training MSE:** 0.625

---

**Initial v:** 1

**After one update:** 1.5

**After two updates:** 1.75

**Final training MSE:** 0.15625

---

**Initial v:** 3

**After one update:** 2.5

**After two updates:** 2.25

**Final training MSE:** 0.15625

---

The equal losses for starts 1 and 3 arise because they are equally distant from 2. They do not have equal final parameters. This simple convex example does not show the multiple solutions or inactive units that

can occur in larger networks.

**List the given facts and one fact that is not supplied.**

---

---

---

### 3. See how it works

A pseudorandom generator produces a sequence determined by its state. In the same Python setup, recreating a local generator with the same seed and making the same calls reproduces the sequence below. Saving the generated values also removes ambiguity about which draw became which parameter.

```
from random import Random

def initial_values(seed):
    rng = Random(seed)
    return [rng.uniform(-1.0, 1.0) for _ in range(4)]

def train(v):
    states = [v]
    for _ in range(2):
        v = v - 0.1 * 5 * (v - 2)
        states.append(v)
    return states

first = initial_values(17)
second = initial_values(17)
assert first == second
print(first)

for start in [0.0, 1.0, 3.0]:
    print(start, train(start))
```

The four random numbers illustrate a saved initialisation sequence; they are not used in the three separate scalar training runs. The range  $[-1, 1]$  is a classroom choice, not a general recommendation for network initialisation. An extra random draw before assigning weights shifts which value is used, even if the seed stays the same.

### Connect the picture and the explanation

Point to the input, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

## 4. Compare the cases

### Compare a familiar case with a changed case

The lesson runs starting at 1 and 3 finish with the same training MSE. Give their final  $v$  values and explain the equality without calling the models identical.

**Keep the procedure fixed. Explain what changed in the inputs and result.**

---

---

---

---

---

### Change one thing in the pictured case

Change: Repeat the  $a = 0$  run with identical arithmetic and settings.

Prediction: It again reaches 1 then 1.5 in this deterministic toy calculation.

Reason: Identical initial state and update rules determine the same arithmetic trace.

## 5. Words that unlock the idea

### Word · Meaning

**Word:** Initialisation

**Meaning:** The procedure that assigns model parameter values before training

---

**Word:** Seed

**Meaning:** A value used to initialise a pseudorandom generator

---

**Word:** Symmetry

**Meaning:** An equivalence between parts of a model that can keep their computations identical

---

**Word:** Reproducibility record

**Meaning:** Saved data settings code and results that let another person check how an experiment was performed

---

Use two terms to explain a step in this week's example.

---

---

---

## 6. Follow a worked example

### Identical units receive identical updates

Hidden unit · Before the update · After the update

**Hidden unit:** First ReLU unit

**Before the update:**  $u$  is 1 and  $a$  is 1

**After the update:**  $u$  is 0.8 and  $a$  is 0.8

---

**Hidden unit:** Second ReLU unit

**Before the update:**  $v$  is 1 and  $b$  is 1

**After the update:**  $v$  is 0.8 and  $b$  is 0.8

---

In this separate two-unit example there are no biases. At  $x = 1$ , output is 2 and target is 1. Each of the four squared-error partial gradients equals 2. A simultaneous update at rate 0.1 changes every weight from 1 to 0.8 and keeps the units identical.

**For  $x = 1$ , calculate the two-unit output and squared error before and after the shown simultaneous update. Have the units become different?**

---

---

---

---

## Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

## A pictorial worked case: Initialisation and reproducibility

Reproduce two training traces by saving their distinct initial values.

## The facts we will use

$\text{LossJ}(a) = (a-2)^2$ ; gradient  $2(a-2)$ ; rate 0.25; two updates. Restart from  $a = 0$  and separately  $a = 4$ .

All inputs and outcomes in this worked example are invented classroom data; no real model run is claimed.

## Initialisation and reproducibility

### Start with the supplied facts

Follow each arrow. Ask which fact allows the next step.



#### 1. Start from 0

Updates:  $a = 1$  then 1.5; final loss 0.25.



#### 2. Start from 4

Updates:  $a = 3$  then 2.5; final loss 0.25.



#### 3. Record experiment

Save initial values, rate, objective and update count; the same final loss hides different parameter paths.

### CHANGE ONE THING

Repeat the  $a = 0$  run with identical arithmetic and settings.

Explain the changed result using the matching lesson.

### Read the picture one step at a time

#### Step · What happens and why

**Step:** 1. Start from 0

**What happens and why:** Updates:  $a = 1$  then 1.5; final loss 0.25.

---

**Step:** 2. Start from 4

**What happens and why:** Updates:  $a = 3$  then 2.5; final loss 0.25.

---

### Step: 3. Record experiment

**What happens and why:** Save initial values, rate, objective and update count; the same final loss hides different parameter paths.

---

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

### Why this result follows

Initialisation supplies the parameter values before learning begins. The same update rule can follow different paths from different starts. Here the two paths end on opposite sides of two and have equal squared distance, so equal loss does not imply equal parameters. Save initial values and settings to reproduce this arithmetic, as well as the final result.

### What this example does not establish

A saved seed alone does not reproduce every software or hardware execution; record environment and nondeterminism too.

## 7. Find and repair a mistake

For the two-unit example,  $y = a \text{ReLU}(ux) + b \text{ReLU}(vx)$ . At  $x = 1$ ,  $u = v = a = b = 1$ , output is 2 and target is 1. Because both ReLUs are in their positive region, changing any one parameter slightly changes  $y$  at rate 1. Squared-error gradient is twice residual times that sensitivity, so each gradient is 2. Students can check it with the symmetric numerical probe from Week 10.

With identical initial weights, identical data and deterministic symmetric updates, these units remain copies and cannot learn distinct roles through those updates alone. Different hidden weights can break the symmetry. For example,  $u = 1$  and  $v = -1$  give hidden activations  $[1, 0]$  at  $x = 1$  and  $[0, 1]$  at  $x = -1$ . This demonstrates different responses, not a guarantee of successful training.

Zero is not always an invalid start:  $v = 0$  worked in our single trained-output-weight model. But setting all four weights to zero in the two-unit product model leaves every single-coordinate loss probe unchanged, because each path still contains a zero factor. Do not generalise the success of one scalar start to every architecture.

### Locate the earliest unsupported step

Both runs use the same seed and generator. Run A assigns the first four draws to its weights. Run B draws a preview number first, then assigns the next four. Must the initial weight arrays match? What should be saved?

**State one repair and the checks you would rerun.**

---

---



---

---

---

---

## 8. Practise together

For the scalar lesson model, start  $v = -2$  and apply two updates at rate 0.1. Give all three  $v$  values and the final training MSE.

**For  $x = 1$ , calculate the two-unit output and squared error before and after the shown simultaneous update. Have the units become different?**

---

---

---

---

**Explain your prediction to a partner before comparing results.**

---

---

---

---

### Practise with less help: Distinguish equality of loss and parameters

Use workbook W2 for the connected example “Initialisation and reproducibility”. First fill the input and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

#### Plan · Do · Check

**Plan:** What is given? Which rule or procedure applies?

**Do:** Record each intermediate step.

**Check:** Does the result follow? What remains unknown?

---

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case.

Each person writes their own explanation.

## Check your reasoning

Use the worked case for Initialisation and reproducibility. Proposed explanation: “The same seed specifies the entire experiment”. Decide whether this explanation is supported. Point to the step or supplied fact that decides; explain your repair if needed.

Answer on your own before discussion. If you are unsure, say which step you need help with.

## Explain the picture

Does the matching final loss mean the parameter values are equal? Point to the relevant step in the picture, then write the changed result and the rule or evidence that supports it.

### My explanation using a pictured step and the given facts

---

---

---

---

This is supported practice. Keep the separately named independent case for an attempt before feedback.

## 9. Run the investigation

### Predict, investigate and preserve the evidence

The lesson runs starting at 1 and 3 finish with the same training MSE. Give their final  $v$  values and explain the equality without calling the models identical.

```
from random import Random

def initial_values(seed):
    rng = Random(seed)
    return [rng.uniform(-1.0, 1.0) for _ in range(4)]

def train(v):
    states = [v]
    for _ in range(2):
        v = v - 0.1 * 5 * (v - 2)
        states.append(v)
    return states
```

```
first = initial_values(17)
second = initial_values(17)
assert first == second
print(first)
for start in [0.0, 1.0, 3.0]:
    print(start, train(start))
```

### Record · Your evidence

**Record:** Given input and fixed rule

**Your evidence:** \_\_\_\_\_

\_\_\_\_\_

**Record:** Prediction before checking

**Your evidence:** \_\_\_\_\_

\_\_\_\_\_

**Record:** Actual result, or “not run”

**Your evidence:** \_\_\_\_\_

\_\_\_\_\_

**Record:** First mismatch and one explanation

**Your evidence:** \_\_\_\_\_

\_\_\_\_\_

**Record:** Boundary or missing input and expected response

**Your evidence:** \_\_\_\_\_

\_\_\_\_\_

On paper, trace the supplied procedure. If code is provided, predict before running it in a local practice file. Never write an expected answer as though it were an observed run.

### Plan → try → check

Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

## 10. Build your understanding

## Explain → apply → challenge

For the scalar lesson model, start  $v = -2$  and apply two updates at rate 0.1. Give all three  $v$  values and the final training MSE.

Both runs use the same seed and generator. Run A assigns the first four draws to its weights. Run B draws a preview number first, then assigns the next four. Must the initial weight arrays match? What should be saved?

A forward-pass program returns the same wrong answer every run with a fixed seed. Has repeatability verified the implementation? Name two stronger checks.

**Explain which evidence supports your conclusion and which claim remains unproved.**

---

---

---

---

---

## 11. Connect your project

Add a labelled initialisation and reproducibility evidence sheet to Image-pattern Network Investigation. Show how you can trace how starting values affect a fixed training budget. Use the supplied fictional case and keep its inputs, intermediate steps, calculation or prediction, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

### Project evidence · Record

**Project evidence:** Component and version

**Record:** \_\_\_\_\_

\_\_\_\_\_

**Project evidence:** Inputs permitted at prediction time

**Record:** \_\_\_\_\_

\_\_\_\_\_

**Project evidence:** Trace and comparison with earlier work

**Record:** \_\_\_\_\_

\_\_\_\_\_

**Project evidence:** Failure, limit and human review

**Record:** \_\_\_\_\_

\_\_\_\_\_

## Evidence for your project

For your Image-pattern Network Investigation, save the input, procedure, observed result and limitation of this check. Label this worked example as practice; use a different, previously unreviewed case when checking independent understanding.

## 12. Show what you know

Save actual initial parameters, their order, data and split IDs, preprocessing rules, model formulas, loss convention, learning rate, batch order, update budget, selection policy, code version, software environment and final checkpoint. A seed helps replay one source of randomness; it does not specify all of these choices or guarantee identical results across different numerical environments.

**Two runs use seed 17 but have different final losses. One run made an extra random draw and used a different batch order. Is either difference evidence that the network is broken? Describe a useful investigation.**

---

---

---

---

Repeatability alone cannot prove correctness: a program can repeat the same mistake. Preserve hand-calculated checks as well as logs. When comparing starts, report every planned run instead of selecting a lucky result using final test outcomes. Next week locates shape and gradient errors.

A forward-pass program returns the same wrong answer every run with a fixed seed. Has repeatability verified the implementation? Name two stronger checks.

**My limited claim and a useful next check**

---

---

---

---

## 13. Study a complete case

## Complete evidence record

### Identical units receive identical updates

Hidden unit · Before the update · After the update

**Hidden unit:** First ReLU unit

**Before the update:** u is 1 and a is 1

**After the update:** u is 0.8 and a is 0.8

---

**Hidden unit:** Second ReLU unit

**Before the update:** v is 1 and b is 1

**After the update:** v is 0.8 and b is 0.8

---

In this separate two-unit example there are no biases. At  $x = 1$ , output is 2 and target is 1. Each of the four squared-error partial gradients equals 2. A simultaneous update at rate 0.1 changes every weight from 1 to 0.8 and keeps the units identical.

Follow the input through the stated procedure to its result. Cover the result first, predict it, then explain every intermediate step. The text/table accompanies the figure so the example can be read without colour.

**My complete trace of the worked case**

---

---

---

---

---

---

## 14. Try a new case independently

### Independent case: Transfer the starting-value experiment independently

Use  $J(a) = (a - 3)^2$ , gradient  $2(a - 3)$ , rate 0.25 and two updates. Restart from  $a = 1$ , then restart from  $a = 5$ . Calculate both traces and final losses. Explain what can be concluded about these two runs.

**My independent explanation, trace and limits**

---

---

---

---

---

---

---

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

### Attempt this without hints

Use  $J(a) = (a - 3)^2$ , gradient  $2(a - 3)$ , rate 0.25 and two updates. Restart from  $a = 1$ , then restart from  $a = 5$ . Calculate both traces and final losses. Explain what can be concluded about these two runs.

#### My independent procedure, result and limitation

---

---

---

---

---

---

---

---

Record support used: none, hint or teacher explanation. After feedback, a different case is needed to demonstrate independence.

### Use feedback to improve

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed. A later check uses a changed case, not a copied answer.